Project Announcement: Data Structures & Algorithms C++ Term Project

Submission Deadline: December 4, 2026
Team Structure: Groups of **3 to 4 students** (Self-select or assigned by lead)

Overview & Guidelines

You will design and implement a C++ project applying core Data Structures and Algorithms. Choose **ONE** project from the list below. Your system must be fully functional, well-documented, and ready for live demonstration/source code review.

List of Project Topics

1. **Student Record Management System**
   - **DSA:** Linked List + File I/O
   - **Core Mechanics:** Nodes represent students (ID, Name, GPA, Course). Supports insertion, deletion (node bypass), sorting, and persistent file storage.
2. **Library Management System**
   - **DSA:** Binary Search Tree (BST) + Hash Table
   - **Core Mechanics:** Books stored in BST sorted by BookID ($O(\log n)$ search). Hash Table maps StudentID → list of issued books.
3. **Hospital Patient Queue System**
   - **DSA:** Priority Queue (Max-Heap) + Queue (FIFO)
   - **Core Mechanics:** Standard patients use FIFO queue; emergency cases use Max-Heap priority queue based on severity (1–10).
4. **Social Network – Friend Suggestion**
   - **DSA:** Graph + BFS / DFS
   - **Core Mechanics:** Vertices represent users; edges represent friendships. BFS (2 levels deep) generates friend suggestions based on mutual connections.
5. **GPS Map Navigation**
   - **DSA:** Graph + Dijkstra's Algorithm
   - **Core Mechanics:** Nodes represent locations; weighted edges represent distances. Computes shortest path using a priority queue.
6. **Text Editor with Undo/Redo**
   - **DSA:** Stack + Doubly Linked List
   - **Core Mechanics:** Text stored as a doubly linked list of characters. Operations use LIFO Undo/Redo stacks.
7. **E-Commerce Product Search Engine**
   - **DSA:** Trie + Hash Table + Merge Sort
   - **Core Mechanics:** Trie handles prefix search / auto-complete. Hash Table provides fast lookup for price filters; Merge Sort handles product rankings.
8. **File Explorer + Duplicate Finder**
   - **DSA:** N-ary Tree + Hashing

- **Core Mechanics:** Folders represented as an N-ary tree. Content hashing (MD5/hash function) identifies duplicate files across directory paths.

9. **Voting / Auction System**
   - **DSA:** AVL Tree (Self-Balancing BST)
   - **Core Mechanics:** Self-balancing rotations maintain $O(\log n)$ search and insertion performance during high-volume entries.
10. **Maze Solver / Snake Game AI**
    - **DSA:** 2D Array + BFS + Backtracking
    - **Core Mechanics:** Grid pathfinding using BFS for shortest path and recursive backtracking for maze exploration/traversal visualizer.

Grading Criteria (100 Points Total)

Criteria	Weight	Description
1. Functionality	**35%**	All core functions work without crashing. Logic correctly handles edge cases and data persists as required.
2. Non-Functionality	**20%**	System performance, memory management (no leaks), execution speed, and CLI/UI usability.
3. Security	**15%**	Input validation, prevention of buffer overflows/infinite loops, bounds checking, and handling of corrupt file inputs.
4. Source Code Review	**30%**	Code readability, proper C++ OOP design, correct pointer/reference usage, comments, and mastery during the live Q&A.

Required Submission Deliverables

Your final repository/submission package must include:

1. **Executable & Source Code:** Cleanly organized .cpp and .h files with build instructions.
2. **Project Documentation File (DOCUMENTATION.md or PDF):**
   - **Project Overview & Architecture:** Explanation of chosen algorithms and data structures.
   - **Development Tools & Environment:** IDE used (e.g., VS Code, Visual Studio, CLion), compiler version (e.g., GCC/G++ version), and built-in or third-party libraries used.
   - **User Manual / How-to-Run:** Step-by-step setup, compilation command, and testing scenarios.
   - **Time & Space Complexity Analysis:** Big-O notation breakdowns for key functions.